

Algoritmer

Sortering



Agenda

- Algoritmer – **9:45**
- Øvelse simpel algoritme
- Effektivitet og klassificering - **10:20**
- Sorteringsalgoritmer
- Insertion sort
- Øvelse
- Frokost - **11:15**
- Bubble sort - **12:00**
- Øvelse
- Quick sort - **12:45**
- Lineær søgning/binær søgning
- Øvelse

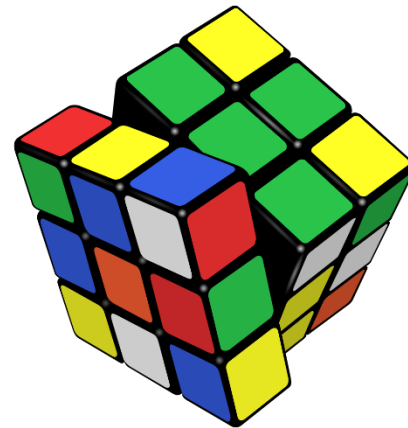


Algoritmer



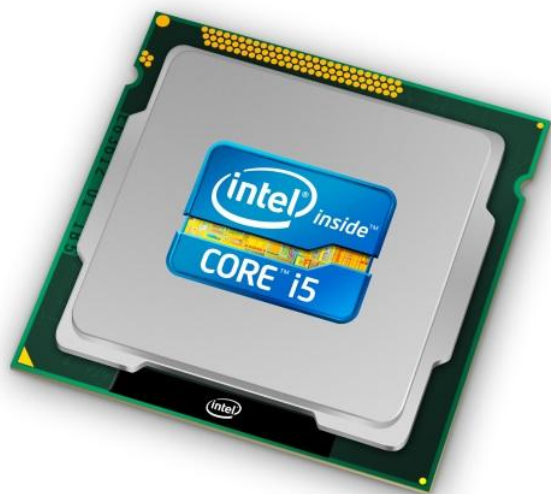
Hvad er en algoritme?

- En algoritme kan ses som
 - En effektiv måde at beregne en løsning til et problem, der kan løses ved hjælp af en række træk.
 - Man kan sige, at det er en opskrift til at løse et bestemt problem.



Hvad er en algoritme?

- Algoritmer danner basis for hvordan en CPU behandler data



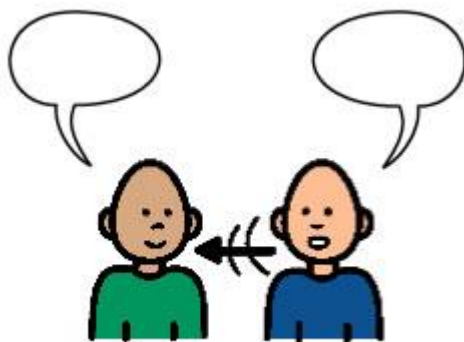
Hvad er en algoritme?

- **Korrekte algoritmer:**
- Afslutter altid og giver en korrekt løsning på problemet.
- **Ikke-korrekte algoritmer:**
- Kan enten:
 - Ikke garantere afslutning (uendelig løkke).
 - Returnere en tilnærmet løsning i stedet for den præcise.
- **Eksempler:**
- Korrekt: A* (A-star)
 - Finder den korteste vej i pathfinding.
- Ikke-korrekt: Monte Carlo Tree Search
 - Brugt i AI til nogle spil, men giver ikke altid den optimale løsning.



Formulering af en algoritme

- En algoritme beskrives ofte med naturligt sprog, men kan også formuleres som pseudokode, flowcharts eller programkode for præcision.



Formulering af en algoritme

- Pseudo kode
 - Konstrueret sprog over hvordan et program kan opbygges
 - Programmeringssproget er ikke fastlagt
 - Kan være en blanding mellem naturligt sprog og programmeringssprog

```
IF (sugar is required)
    THEN add sugar;
    ELSE don't add sugar;
ENDIF
```

```
IF (enemy is visible)
    THEN attack enemy;
    ELSE patrol area;
ENDIF
```

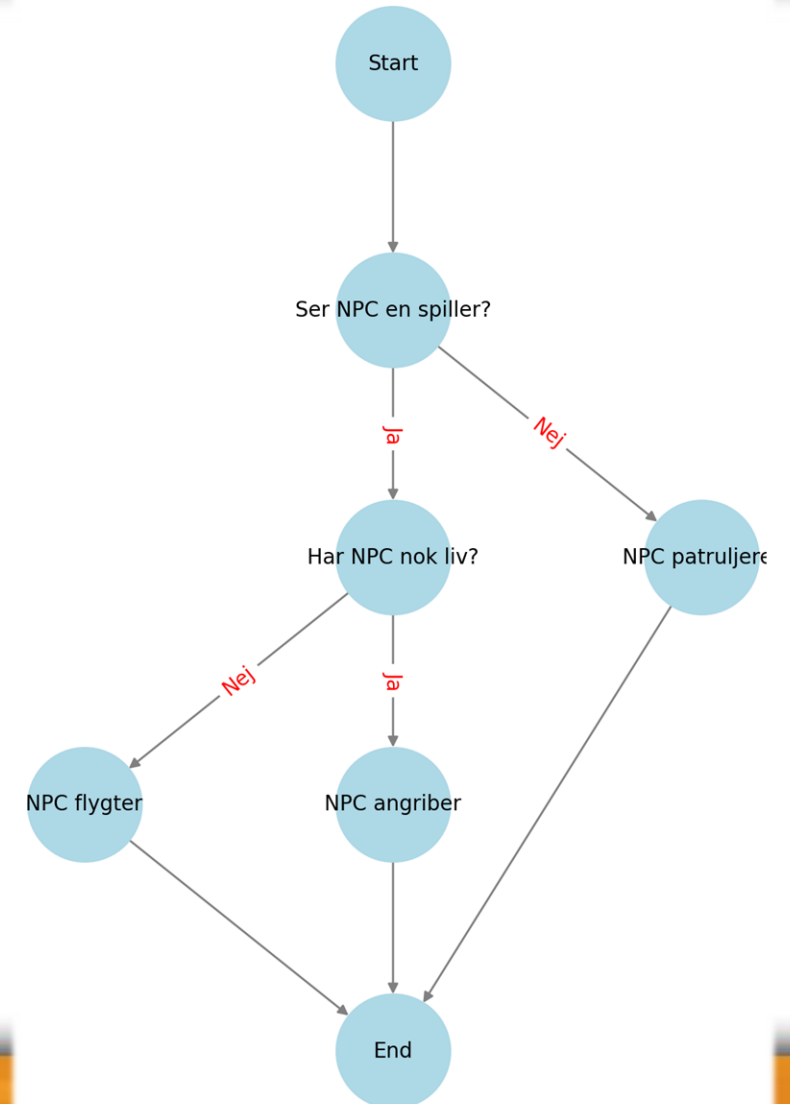


Formulering af en algoritme

- **Flowcharts**

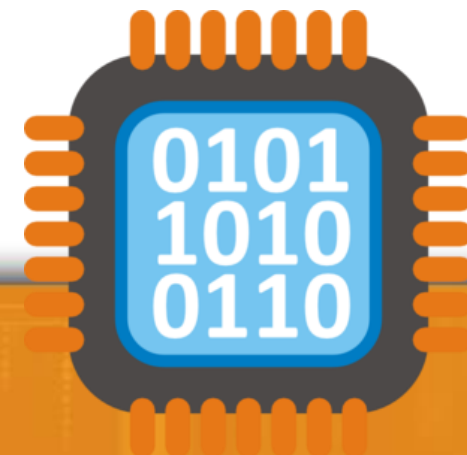
- En **grafisk repræsentation** af en algoritme baseret på valg og konsekvenser.
- Består af **processer, beslutninger og loops**, ofte brugt til at visualisere logik i programmering.
- Hjælper med **forståelse af algoritmer** før implementering i kode.

Flowchart: NPC Beslutningsproces



Formulering af en algoritme

- **Konstruerede sprog (Programmeringssprog)**
 - En algoritme formuleres i et **programmeringssprog**, så den kan udføres af en CPU.
- **Struktur og syntaks** afhænger af sproget (f.eks. Python, C#, Java).
 - Mere præcist end pseudokode, men kræver korrekt syntaks.
- **Fejlhåndtering undlades ofte**
 - Fokus er på logikken, ikke fejl.
 - Fejlhåndtering tilføjes ofte senere.



Formulering af en algoritme

- **En algoritme defineres ved:**
- **Input:** Data, som algoritmen skal arbejde med (f.eks. en liste af tal).

5 6 4 8 3

- **Output:** Det forventede resultat efter beregning (f.eks. en sorteret liste).

3 4 5 6 8

- **Beskrivelse:** De trin, der omdanner input til output.



Eksempel på en algoritme

Største tal i mængden:

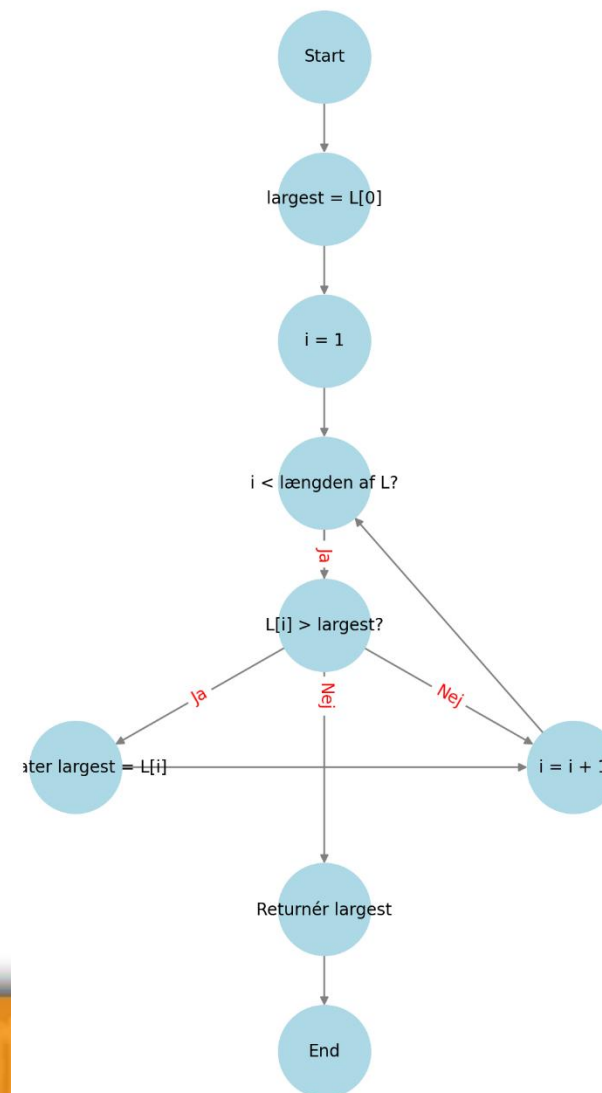
- Højniveau beskrivelse:
 - Antag, at første tal er det største.
 - Gennemgå resten af tallene ét ad gangen.
 - Hvis et tal er større, opdater største tal.
 - Når alle er gennemgået, er det største tal fundet.



Eksempel på en algoritme

- Største tal i mængden
 - Formel / Pseudo beskrivelse
 - Algoritme: LargestNumber
 - Input: En ikke-tom liste af tal, L
 - Output: Det største tal i L
-
- Beskrivelse:
 1. Sæt $\text{largest} = L[0]$
 2. Gennemgå resten af L:
 - Hvis $L[i] > \text{largest}$, opdater largest
 3. Returnér largest

Flowchart: Find største tal i en liste



Øvelse

øvelse 031

- Implementer algoritmen som finder det største tal i mængden
 - Lav først en metode til algoritmen
 - Anvend algoritmen på mængde af 10 tilfældige tal
- Implementer en algoritme som finder alle ulige tal i en mængde
 - Lav først en metode til algoritmen
 - Anvend algoritmen på mængde af 10 tilfældige tal

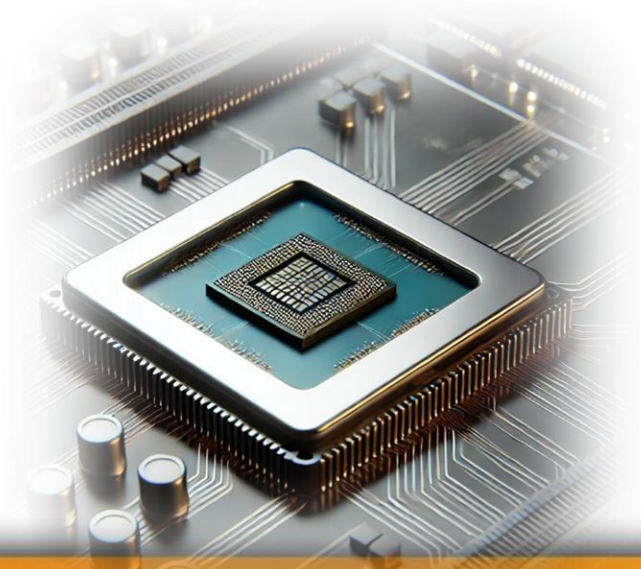


Effektivitet og klassificering



Effektivitet

- **Hvorfor er effektivitet vigtig?**
 - CPU-tid og hukommelse er begrænsede ressourcer.
 - Flere algoritmer kan løse samme problem – men hvilken er bedst?
- **Spørgsmål at overveje:**
 - Hvilken algoritme er hurtigst?
 - Hvilken bruger mindst hukommelse?
 - Hvilken giver det mest præcise resultat?



Effektivitet

- Effektivitet afhænger af inputstørrelsen, hvor n angiver, hvordan algoritmen skalerer.
- Big O-notation (bør i være bekendte med allerede)
 - beskriver worst-case scenariet for væksten i beregningstid.
- Eksempler inkluderer
 - $O(1)$ for konstant tid
 - $O(\log n)$ for logaritmisk vækst
 - $O(n)$ for lineær vækst
 - $O(n^2)$ for kvadratisk vækst.
- Jo lavere O-værdi, desto mere effektiv er algoritmen.
 - Tjek eventuelt dette link: [bigocheatsheet.com](https://www.bigocheatsheet.com/).

<https://www.bigocheatsheet.com/>



Effektivitet

- **O (Big O)**

- Øvre grænse (**Upper bound**)
- **Worst case** – Maksimal tid en algoritme kan tage

- **Ω (Omega)**

- Nedre grænse (**Lower bound**)
- **Mindste tid** en algoritme kan tage

- **Θ (Theta)**

- **Gennemsnitlig vækst (Tæt grænse)**
- Algoritmen vokser ikke hurtigere eller langsommere end dette



Effektivitet

- Big-O (store O) eksempel med kode:
 - Nedenstående kode løber en liste igennem og returnerer true, hvis vi finder det vi søger
 - $O(n)$ (*Lineær tidkompleksitet*)
 - *effektiv til små datasæt, men skalerer dårligt til store.*

```
public static bool ExistInList(List<Person> people, string name)
{
    foreach (Person person in people)
    {
        if (person.Name == name)
        {
            return true;
        }
    }

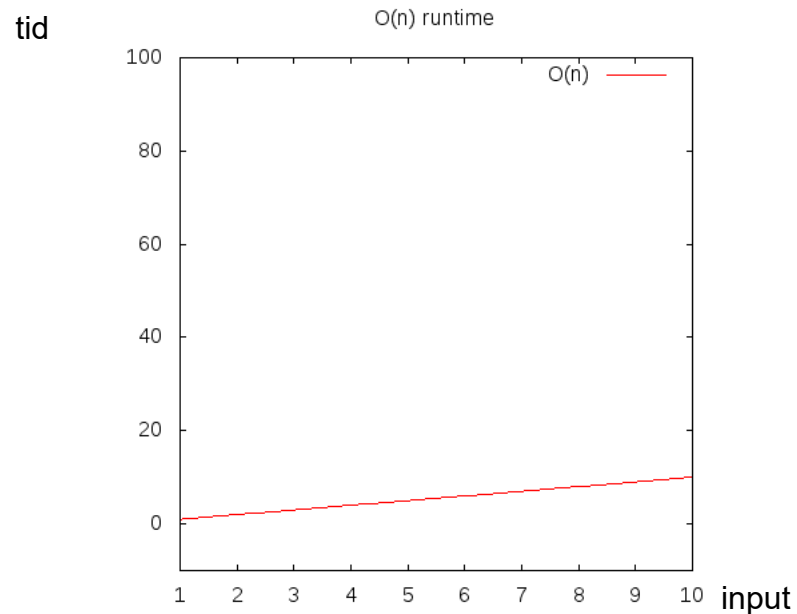
    return false;
}
```



Effektivitet

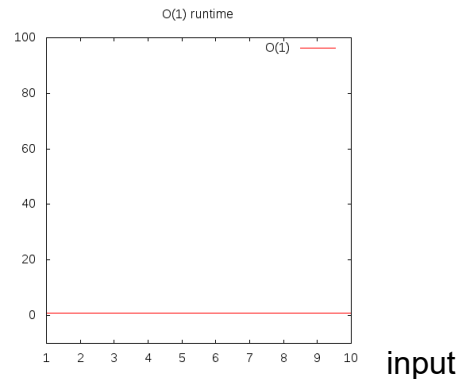
- **$O(n)$ – Lineær tidskompleksitet**

- Køretiden **stiger lineært** med antallet af elementer i listen.
- **Worst case:** Det søgte element er sidst i listen – eller findes slet ikke.
- **Lineær søgning** gennemgår hvert element én for én, indtil det rigtige findes.



Effektivitet

- $O(1)$
 - Constant time, lige meget hvor stort vores input er, så tager det altid samme tid at udføre opgaven



```
public static Person FindOnIndex(List<Person> people, int index)
{
    return people[index];
}
```



Effektivitet

- **$O(n^2)$ – Kvadratisk tid**

- Beregningstiden vokser **hurtigt**, når input bliver større.
 - **Eksempel:** 1 element = 1 sek, 10 elementer = 100 sek.
 - **Eksempel på algoritme:** *Bubble Sort*.

- **$O(\log n)$ – Logaritmisk tid**

- Beregningstiden vokser **langsomt**, selv ved store input.
 - **Eksempel:** 1024 elementer tager kun **dobbelt så lang tid** som 32 elementer.
 - **Eksempel på algoritme:** *Binær søgning*, *binært søgetræ*.

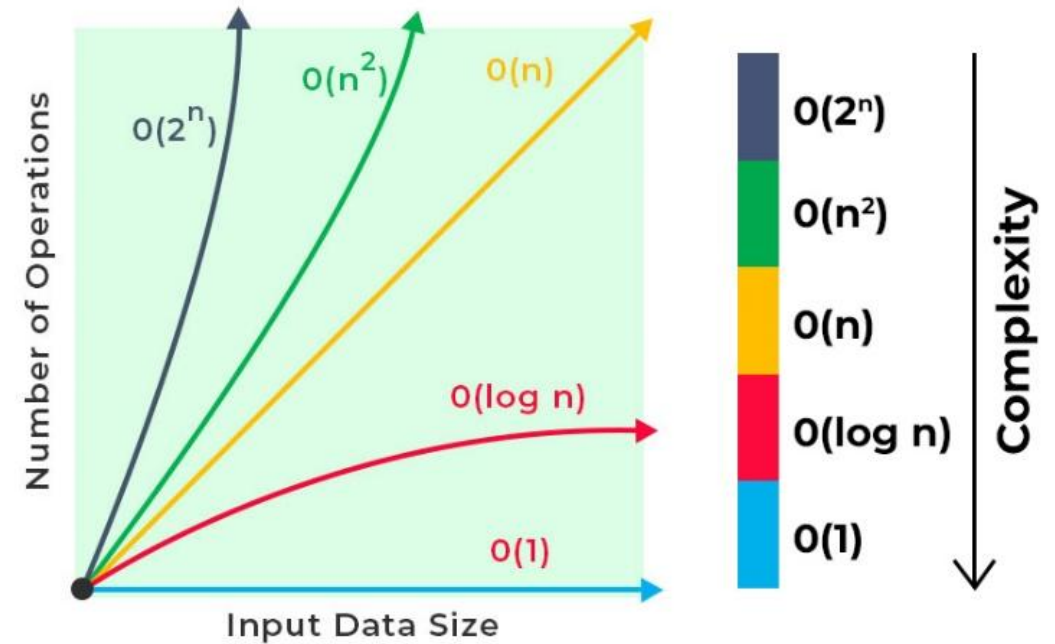
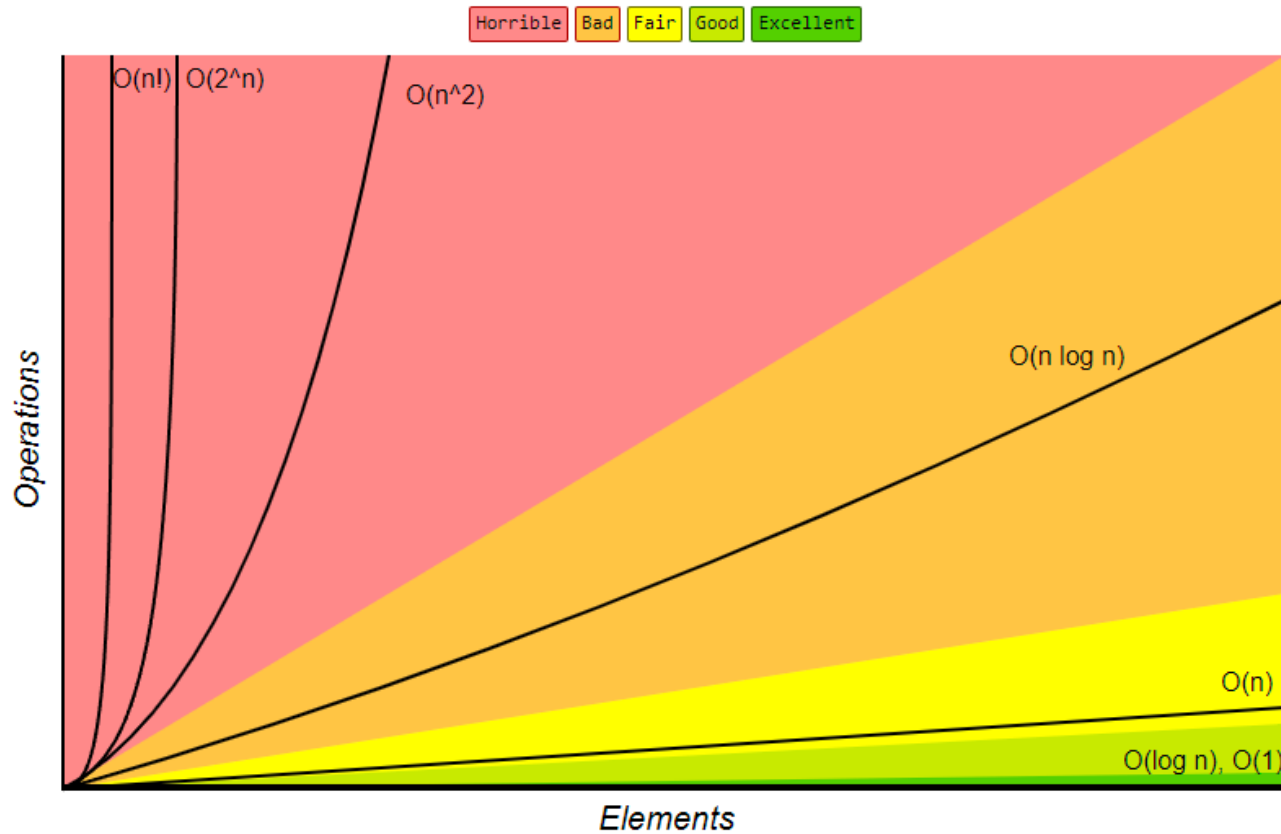
- **$O(n \log n)$ – Hurtigere sortering**

- Bruges i **effektive sorteringsalgoritmer**.
 - **Eksempel på algoritme:** *Quicksort*, *Heap Sort*.



Effektivitet

Big-O Complexity Chart



Klassificering af algoritmer

- Der findes mange forskellige måder at klassificering en algoritme på
- Overordnet gør klassificeringer os i stand til
 - at indgå i dialoger med andre fagfolk
 - at tage beslutninger om vores behov
- Algoritme klassificeringerne hjælper os med af forstå:
 - Opbygningen af algoritmen (Implementering)
 - Måden hvorpå den løser problemet (Design paradigmet)
 - Formålet (Akademisk felt)
 - Effektiviteten (Kompleksiteten)



Klassificering af algoritmer

- **Implementeringer af algoritmer:**

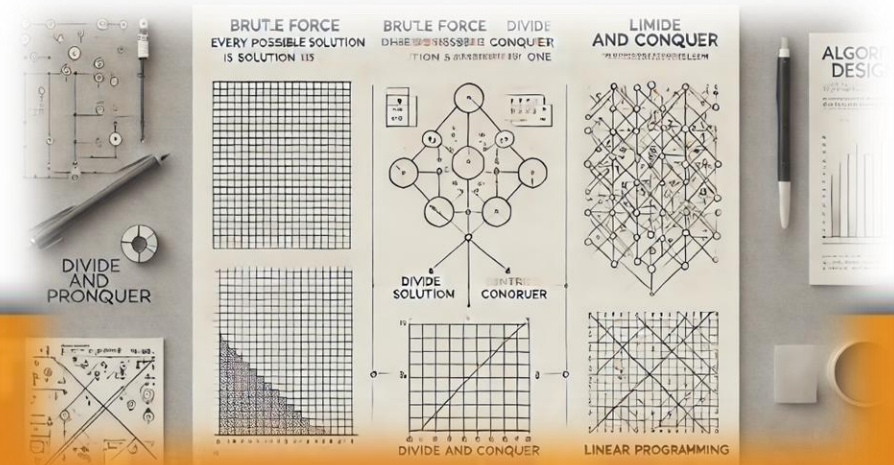
- **Rekursiv / Iterativ** – En algoritme kan enten **kalde sig selv gentagne gange (rekursion)** eller **køre i løkker (iteration)**.
- **Logiske** – Algoritmen følger et **fastlagt logisk forløb**, styret af kontrolstrukturer.
- **Sekventiel / Parallel / Distribueret** – Kører enten **én instruktion ad gangen (sekventiel)**, **flere instruktioner samtidig (parallel)** eller **på flere forbundne computere (distribueret)**.
- **Deterministisk / Ikke-deterministisk** – En deterministisk algoritme producerer **altid det samme output**, mens en ikke-deterministisk algoritme bruger **tilfældige valg eller gæt**.
- **Præcis / Tilnærmelse** – En præcis algoritme giver **det korrekte svar**, mens en tilnærmelsesalgoritme finder en **næsten optimal løsning**.



Klassificering af algoritmer

- **Design paradigme:**

- **Brute Force / Udtømmende søgning** – Afprøver **alle mulige løsninger** for at finde den bedste eller korrekte.
- **Divide and Conquer** – **Deler problemet op i mindre dele**, som løses individuelt og kombineres til en løsning.
- **Lineær programmering** – Beregner den **optimale løsning** for et matematisk problem ved at maksimere eller minimere en funktion.



Klassificering af algoritmer

- **Designparadigme:**

- **Reduktion / Transform and Conquer**
- Omdanner problemet til et **kendt problem**, som er lettere at løse.
- **Eksempel:** Sortering hvor problemet **opdeles i mindre, lettere håndterbare dele**.

- **Search and Enumeration**

- **Bruger grafer** til at modellere problemet og finder løsningen ved at gennemgå grafen.
- **Bruger heuristikker** eller erfaring til at vælge den bedste vej.
- **Eksempel: Dijkstra's algoritme** til korteste vej.



Klassificering af algoritmer

- **Akademisk emnefelt & Komplexitet**

- **Akademisk emnefelt**

- Algoritmer klassificeres efter **brugsområde**.
 - **Eksempler:** Søgning, sortering, numeriske beregninger, grafalgoritmer.

- **Komplexitet**

- Beskriver, **hvor meget tid** en algoritme bruger baseret på inputstørrelsen.
 - **Eksempel:** $O(n)$ – Lineær tid, hvor tiden stiger proportionalt med inputmængden.
 - Måles ofte med: **Big O-notation**.



Sorteringsalgoritmer



Sorteringsalgoritmer

- Nogle af de mest anvendte algoritmer
 - Anvendes tit som et mellemstredt i programmer eller andre algoritmer
- Generelle krav
 - Outputtet er en omstrukturering af inputtet
 - Ikke nyt data
 - Der må ikke oprettes nye objekter, de originale skal bibeholdes
 - Ingen manipulation af data



Insertion sort

- Gennemløber en mængde gentagende gange
 - Sammenligner elementerne
 - Bytter elementerne hvis de er i forkert rækkefølge
- Minder om den måde man tager kort op på hånden i et kortspil

<https://www.youtube.com/watch?v=8mJ-OhcfpYg>



Insertion sort

- Algoritmen:

- **Input:** En ikke tom mængde af tal, kaldet L f.eks. $\langle L_0, L_1, \dots, L_n \rangle$
- **Output:** Sorteret mængde af tal $\langle L_0, L_1, \dots, L_n \rangle$ således $L_0 \leq L_1 \leq \dots \leq L_n$

- Pseudokode:

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

Insertion Sort

1. For hvert element i listen (undtagen det første):
 - a. Gem det aktuelle element.
 - b. Flyt elementer til højre, indtil det korrekte indsætningspunkt er fundet.
 - c. Indsæt det gemte element på den korrekte position.



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val <
    L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

for i = 1 to length(L) - 1 do

Start her



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val <
    L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

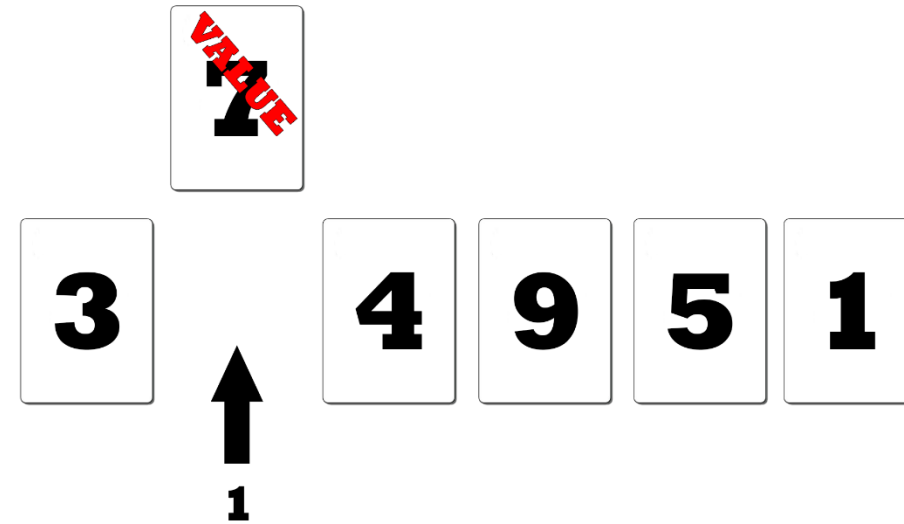
val = L[i]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val <
    L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

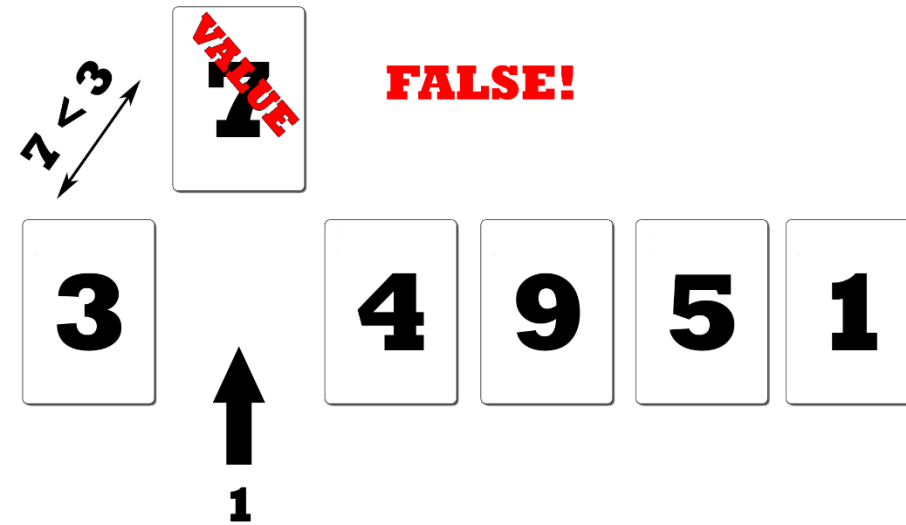
pointer = i



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val <
    L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

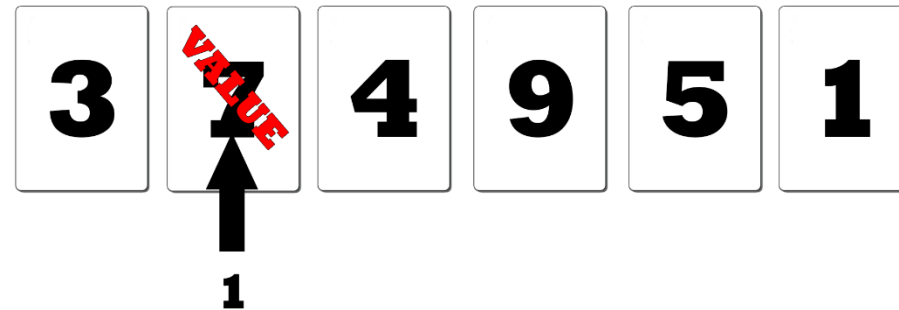
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val <
    L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

L[pointer] = val

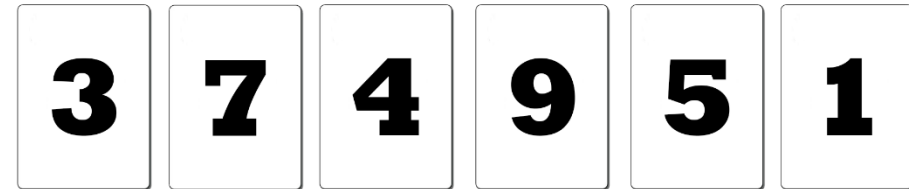


Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

for i = 1 to length(L) - 1 do

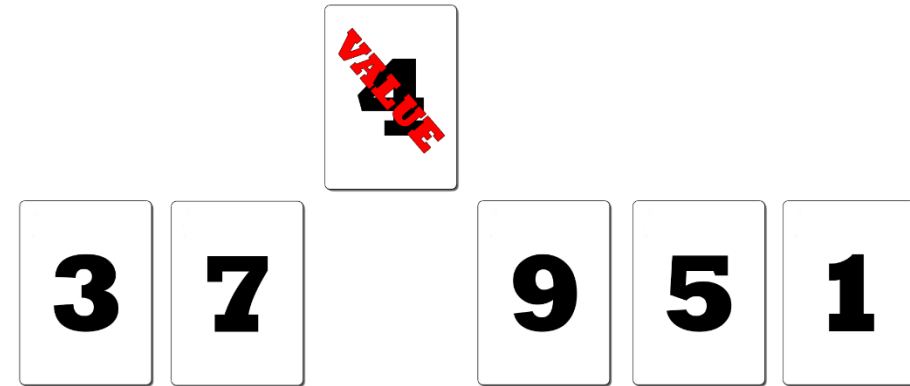
Fortsæt her



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

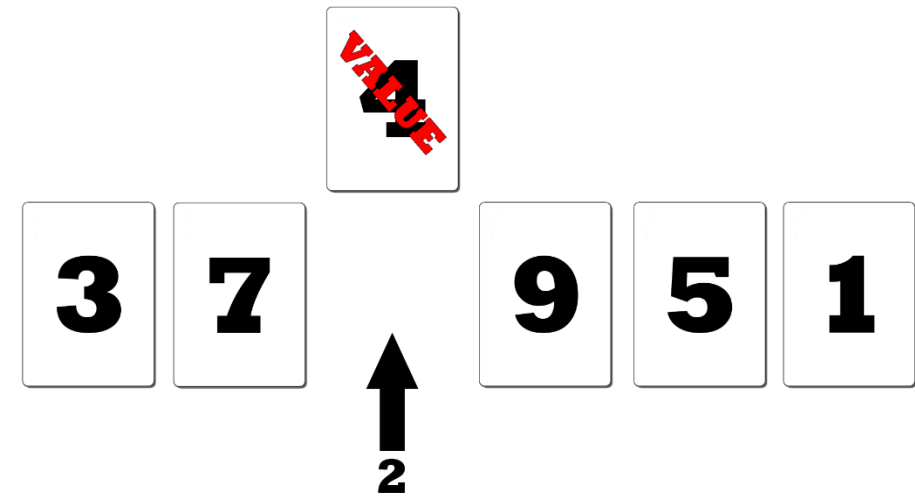
val = L[i]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

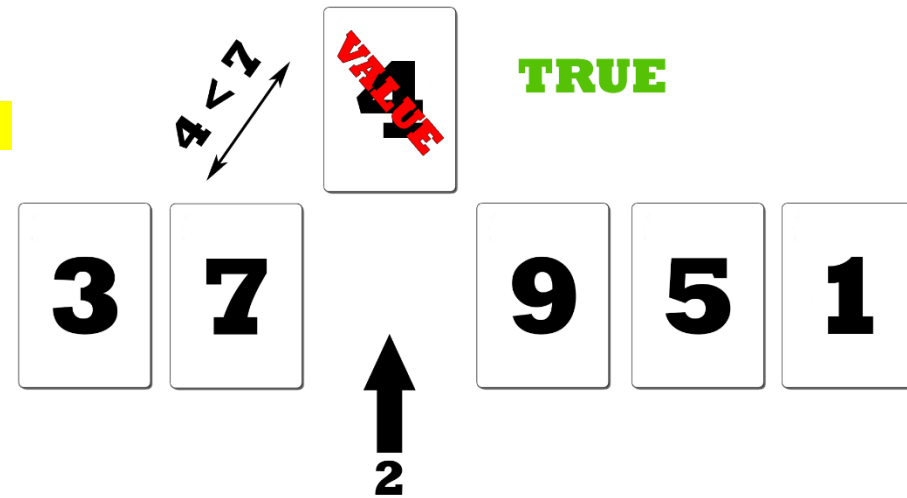
pointer = i



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

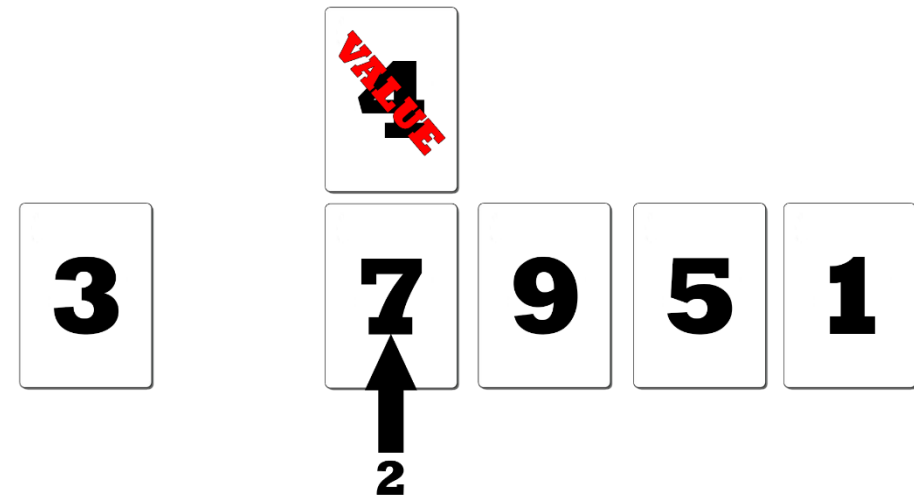
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

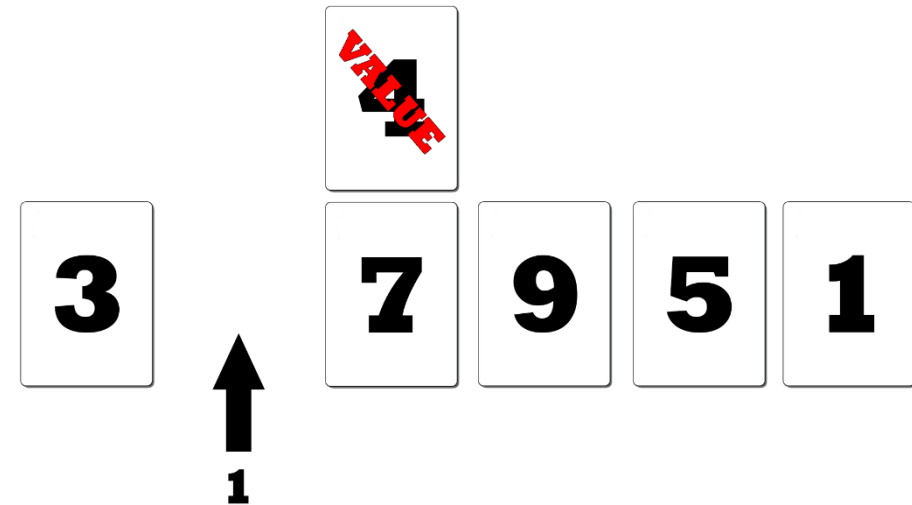
L[pointer] = L[pointer - 1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

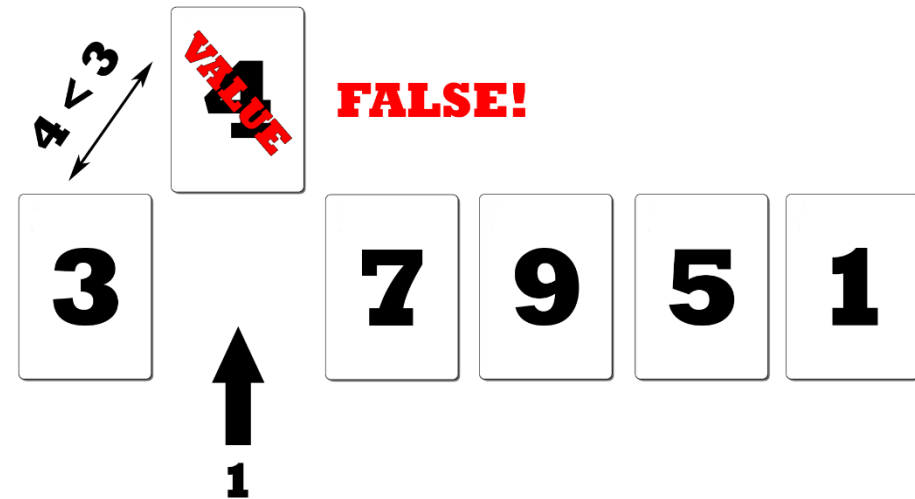
point = pointer - 1



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

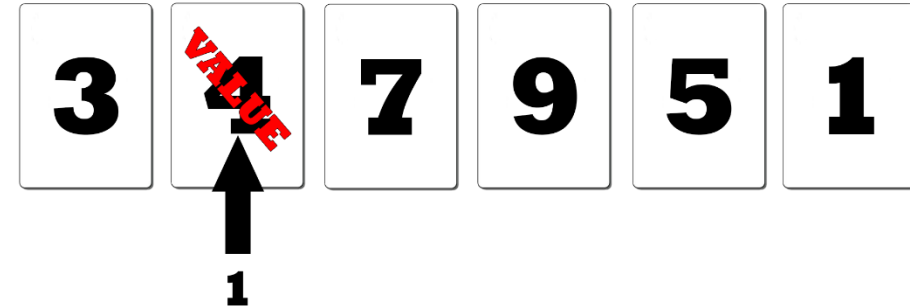
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

L[pointer] = val

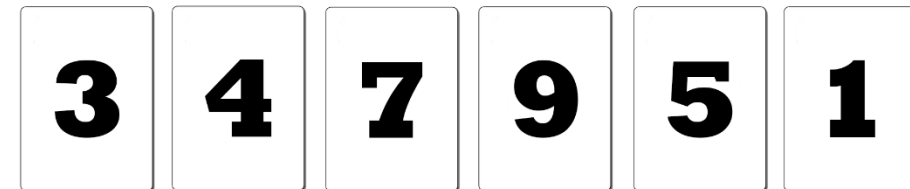


Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

for i = 1 to length(L) - 1 do

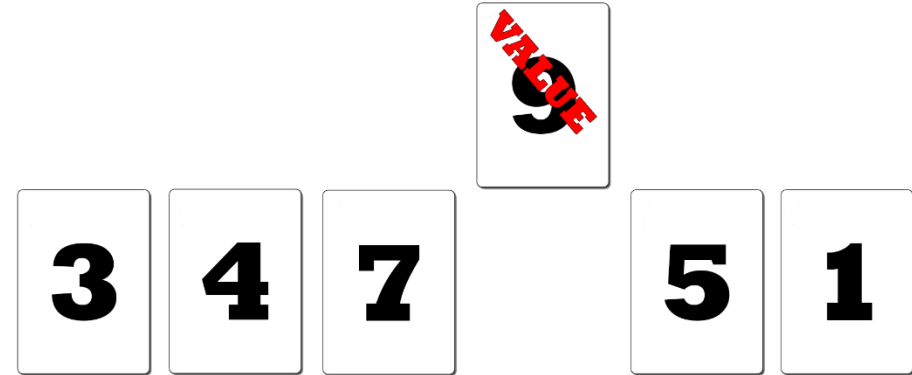
Fortsæt her



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

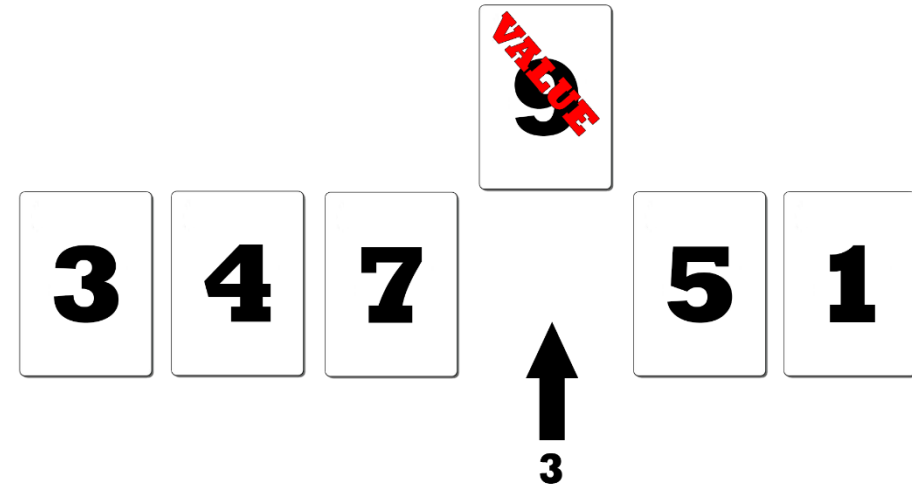
val = L[i]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

pointer = i



Insertion sort

```
for i = 1 to length(L) - 1 do
```

```
  val = L[i]
```

```
  pointer = i
```

```
  while pointer > 0 and val < L[pointer - 1]
```

```
    L[pointer] = L[pointer - 1]
```

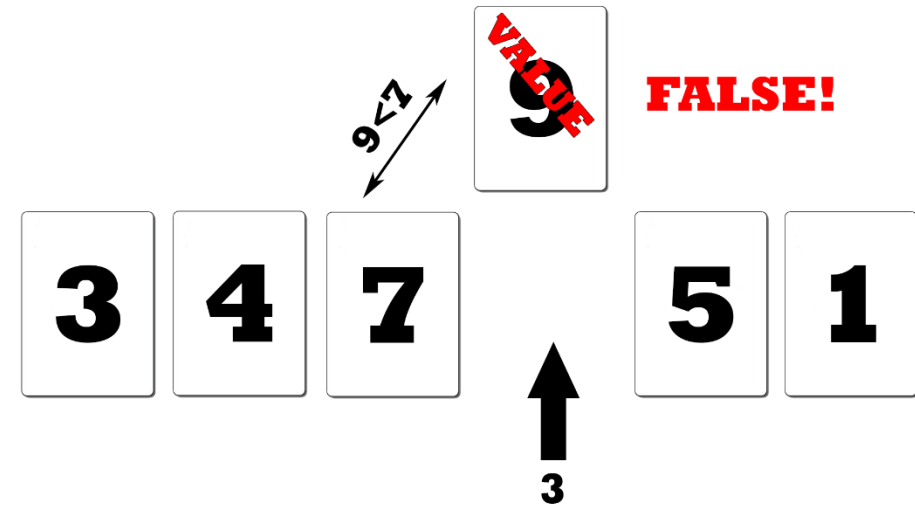
```
    pointer = pointer - 1
```

```
  end while
```

```
  L[pointer] = val
```

```
end for
```

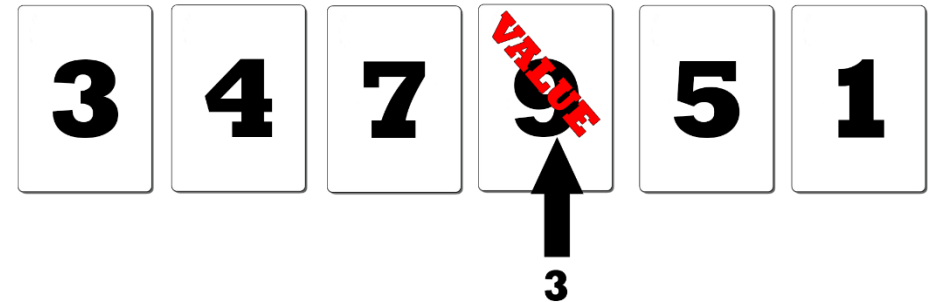
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

L[pointer] = val



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

for i = 1 to length(L) - 1 do

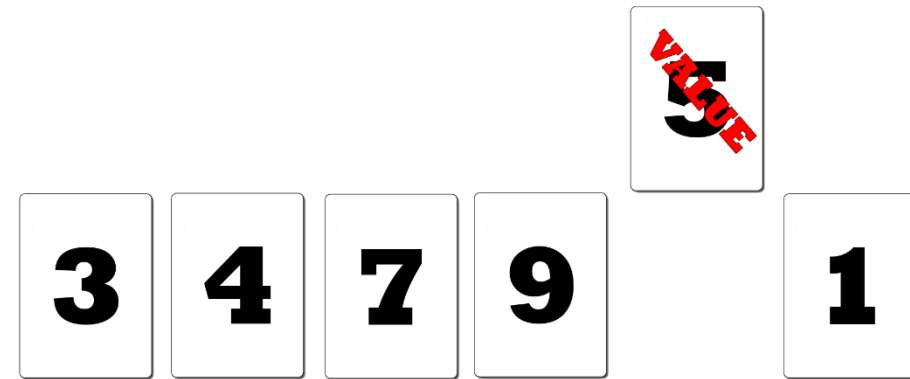
Fortsæt her



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

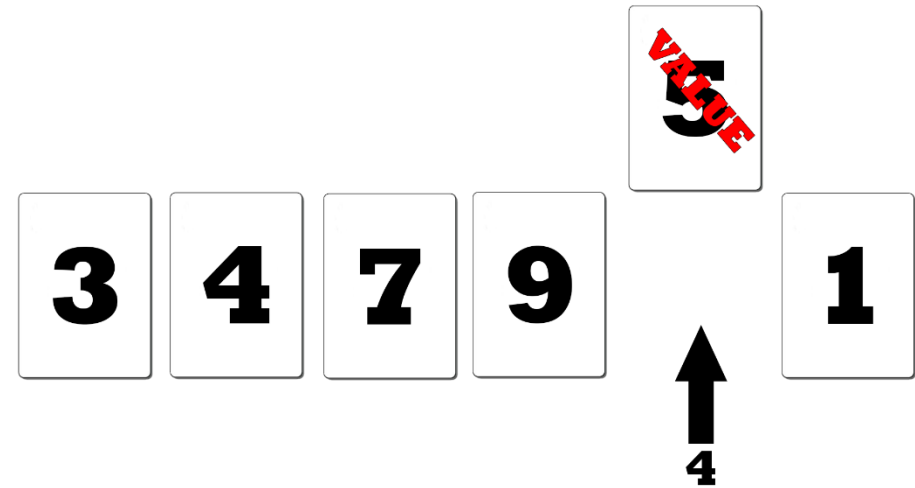
val = L[i]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

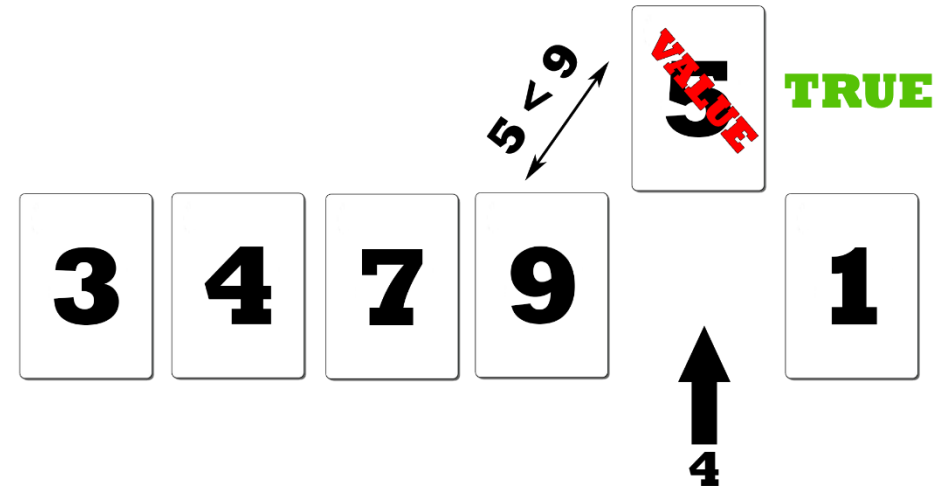
pointer = i



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

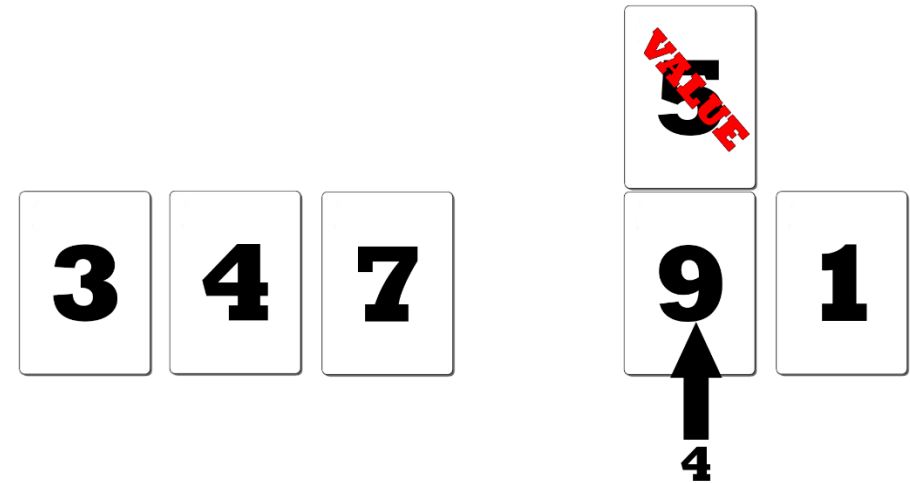
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

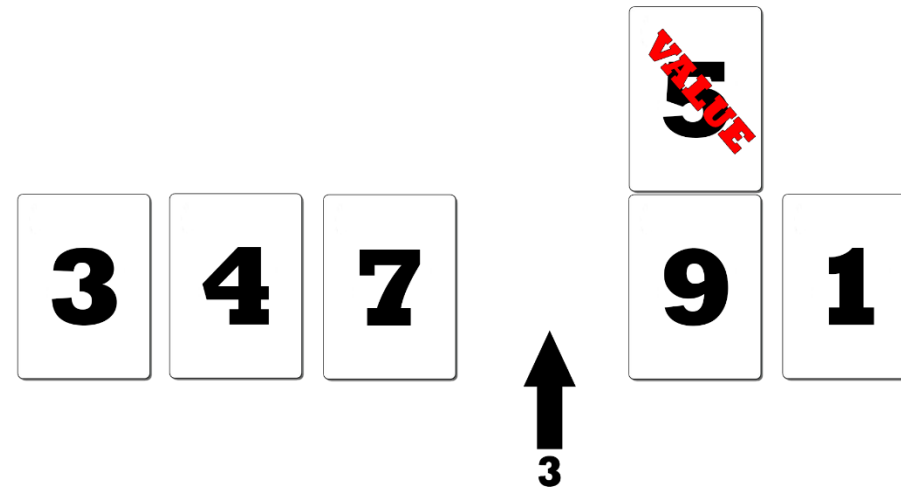
L[pointer] = L[pointer - 1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

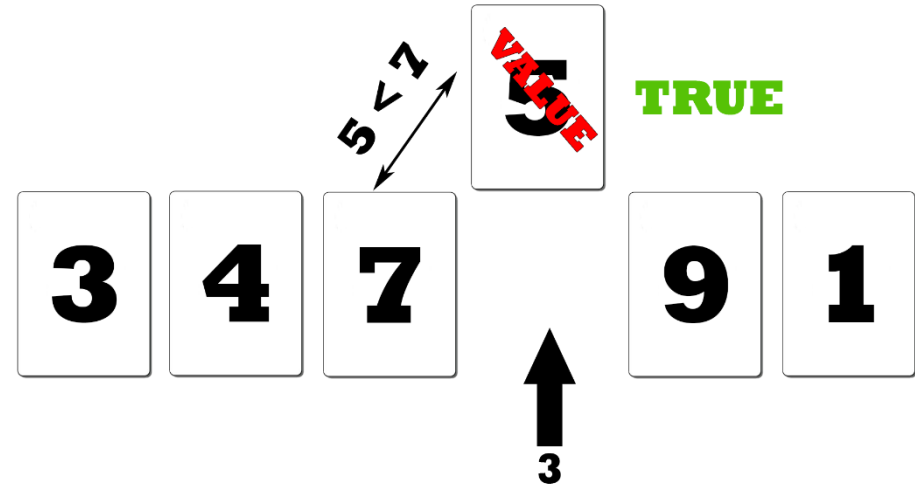
point = pointer - 1



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

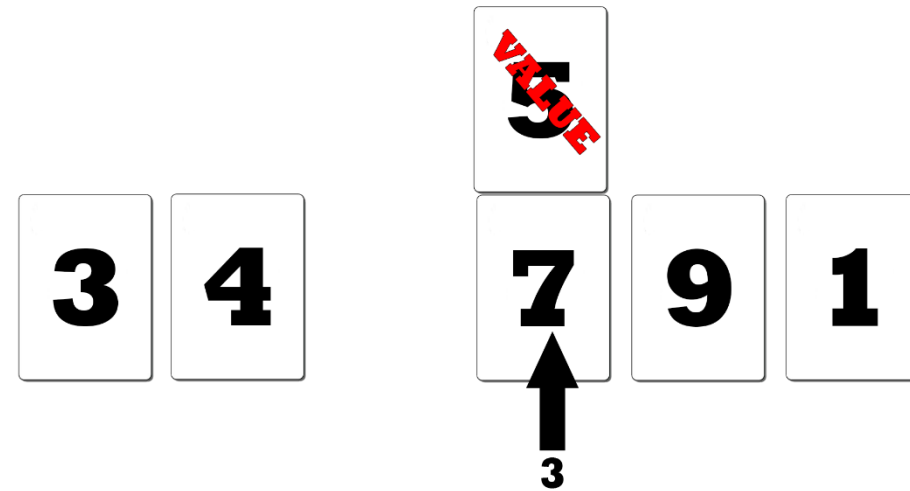
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

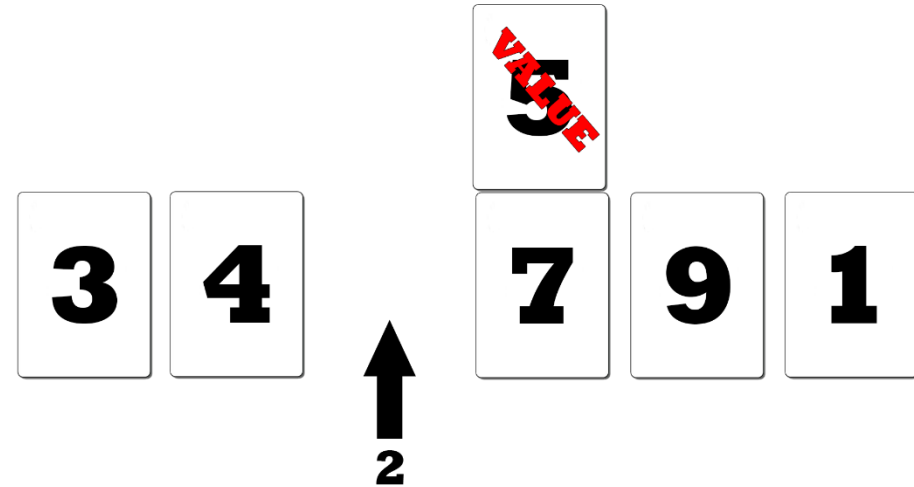
L[pointer] = L[pointer - 1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val <
    L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

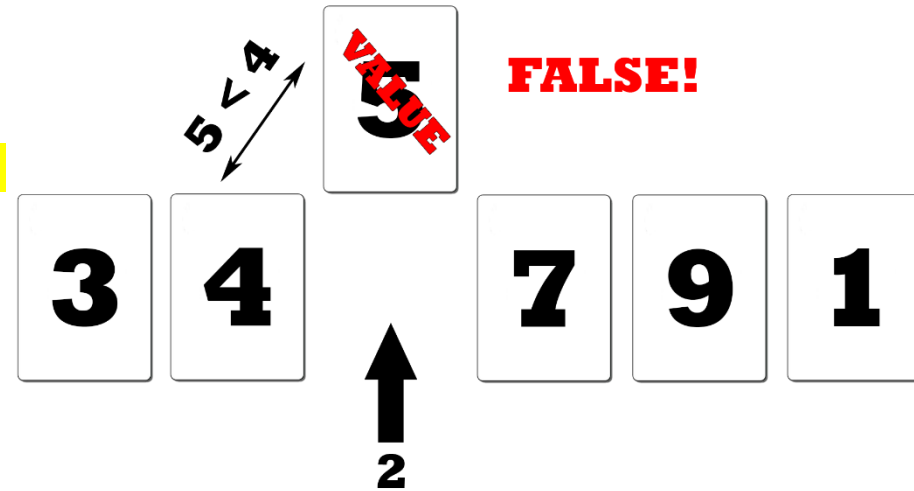
point = pointer - 1



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

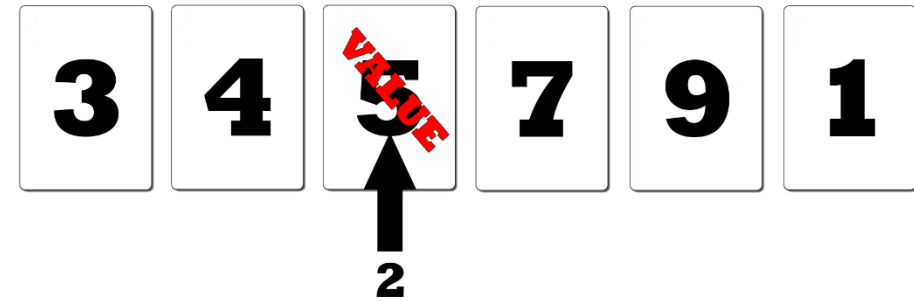
while pointer > 0 and val < L[pointer-1]



Insertion sort

```
for i = 1 to length(L) - 1 do
  val = L[i]
  pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

L[pointer] = val

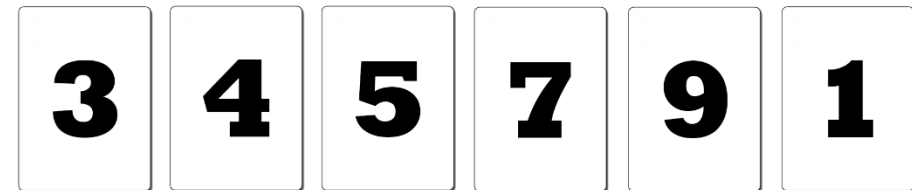


Insertion sort

```
for i = 1 to length(L) - 1 do
  tmp  val = L[i]
  j    pointer = i
  while pointer > 0 and val < L[pointer - 1]
    L[pointer] = L[pointer - 1]
    pointer = pointer - 1
  end while
  L[pointer] = val
end for
```

for i = 1 to length(L) - 1 do

Fortsæt her



1 bliver rykket hele vejen til starten af listen!



Insertion sort

- **Sådan virker den (kort forklaring)**

1. Gå gennem listen ét element ad gangen.
2. Tag det aktuelle element og sammenlign det med de tidligere elementer.
3. Flyt større elementer ét skridt til højre for at gøre plads.
4. Indsæt elementet på den rigtige plads.
5. Gentag, indtil hele listen er sorteret.



Insertion sort

- Et eksempel med listen **[5, 3, 4, 1, 2]**
 - Første element: [5]
 - betragtes alene og er allerede sorteret.
 - Næste element: [3]
 - sammenlignes med 5 og indsættes foran, da det er mindre. Listen er nu **[3, 5]**.
 - Næste element: [4]
 - sammenlignes med 5 og placeres mellem 3 og 5. Listen er nu **[3, 4, 5]**.
 - Næste element: [1]
 - er det mindste og flyttes derfor forrest i listen. Listen er nu **[1, 3, 4, 5]**.
 - Sidste element [2]
 - sammenlignes og placeres mellem 1 og 3. Listen er nu **[1, 2, 3, 4, 5]**.



Insertion sort

Effektivitet af Insertion Sort

- **Best case: $\Omega(n)$** – Listen er allerede sorteret, så hvert element indsættes uden flytninger.
- **Average case: $\Theta(n^2)$** – Elementerne er tilfældigt blandet, og mange flytninger er nødvendige.
- **Worst case: $O(n^2)$** – Listen er sorteret i omvendt rækkefølge, så hvert element skal flyttes maksimalt.



Øvelse

- Lav en implementering af Insertion Sort i C# og brug den på et array.



Bubble sort

- En simpel sorteringsalgoritme, der sammenligner nærliggende elementer og bytter dem, hvis de er i forkert rækkefølge.
 - Gentager processen, indtil listen er sorteret.
 - Nem at implementere, men ineffektiv for store lister.
 - Har en tidkompleksitet på $O(n^2)$.

<https://www.youtube.com/watch?v=Dv4qLJcxus8&t=15s>



Bubble sort

- Algoritmen:
 - **Input:** En ikke tom mængde af tal, kaldet L f.eks. $\langle L_0, L_1, \dots, L_n \rangle$
 - **Output:** Sorteret mængde af tal $\langle L_0, L_1, \dots, L_n \rangle$ således $L_0 \leq L_1 \leq \dots \leq L_n$
- Pseudokode:

```
procedure bubbleSort( L : list of sortable items )
Repeat
    swapped = false
    for i = 1 to length(L) - 1 inclusive do:
        if L[i-1] > L[i] then
            swap(L[i-1], L[i])
            swapped = true
        end if
    end for
Until not swapped
Return L
```

Bubble Sort

1. Gentag indtil listen er sorteret:
 - a. Gå gennem listen og sammenlign naboelementer.
 - b. Hvis et element er større end det næste, byt dem.
 - c. Hvis ingen bytninger blev foretaget, er listen sorteret.



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
        if L[i-1] > L[i] then  
            swap(L[i-1], L[i])  
            swapped = true  
        end if  
    end for  
Until not swapped  
Return L
```

swapped: false

5 1 4 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: false

i = 1

5 1 4 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: false

i = 1

5 1 4 2 8
↑↑

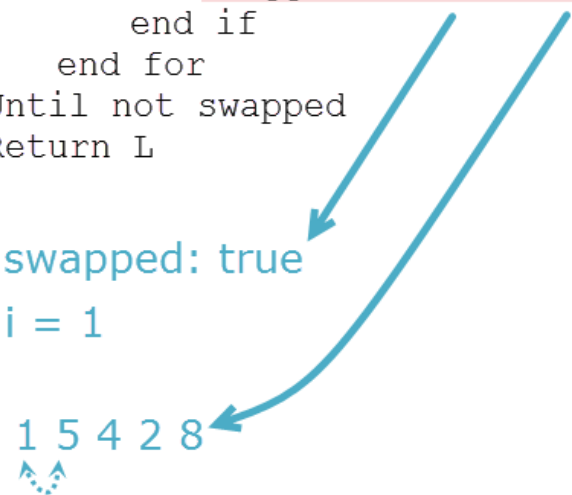
Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 1

1 5 4 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 2

1 5 4 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 2

1 5 4 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 2

1 4 5 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 3

1 4 5 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 3

1 4 5 2 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 3

1 4 2 5 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 4

1 4 2 5 8



Bubble sort

```
procedure bubbleSort( L : list of sortable items )  
  Repeat  
    swapped = false  
    for i = 1 to length(L) - 1 inclusive do:  
      if L[i-1] > L[i] then  
        swap(L[i-1], L[i])  
        swapped = true  
      end if  
    end for  
  Until not swapped  
  Return L
```

swapped: true

i = 4

1 4 2 5 8



False

Bubble sort

- **Første gennemløb:**

(5 1 4 2 8) (1 5 4 2 8), Sammenligner og bytter første to elementer

(1 5 4 2 8) (1 4 5 2 8), Bytter fordi $5 > 4$

(1 4 5 2 8) (1 4 2 5 8), Bytter fordi $5 > 2$

(1 4 2 5 8) (1 4 2 5 8), Ingen bytning fordi $8 > 5$

- **Andet gennemløb**

(1 4 2 5 8) (1 4 2 5 8)

(1 4 2 5 8) (1 2 4 5 8), Bytter fordi $4 > 2$

(1 2 4 5 8) (1 2 4 5 8)

(1 2 4 5 8) (1 2 4 5 8)

Listen er sorteret, men algoritmen skal køre endnu et gennemløb for at vide dette (kan optimeres) pga. andet gennemløb satte swapped = true

- **Tredige gennemløb:**

(1 2 4 5 8) (1 2 4 5 8)

(1 2 4 5 8) (1 2 4 5 8)

(1 2 4 5 8) (1 2 4 5 8)

(1 2 4 5 8) (1 2 4 5 8)



Bubble sort

Effektivitet af Bubble Sort

- **Best case: $\Omega(n)$** – Listen er allerede sorteret, så kun én gennemløbning er nødvendig.
- **Average case: $\Theta(n^2)$** – Elementerne er tilfældigt blandet, og algoritmen skal bytte mange gange.
- **Worst case: $O(n^2)$** – Listen er sorteret i omvendt rækkefølge, så hvert element skal flyttes maksimalt.



Øvelse

- Lav en implementering af BubbleSort i C# og brug den på et array



QuickSort

- Gør brug af divide and conquer princippet
 - Det er denne metode, som anvendes i C# til sortering
 - List<T>
 - Stack<T>
 - Queue<T>
 - Dictionary<T>
 - SortedList<TKey, TValue>
- Algoritmen er ofte lavet rekursivt
 - Dvs. den kalder sig selv flere gange for at indsnævre / løse problemet

<https://www.youtube.com/watch?v=Vtckgz38QHs>



QuickSort

1. Udvælg et element fra mængden kaldet et pivot

7 2 7 9 **[4]** 3 8 6 5 1



2. Sorter elementerne: mindre < pivot < større

[2 1 3] <**4**< [7 7 8 6 5 9]

3. Algoritmen bruges rekursivt på begge mængder



QuickSort

Et nyt pivot vælges i den mindre mængde

[2 [**1**] 3] **4** [7 7 8 6 5 9]

Den mindre delmængde sorteres

[< **1** < 2 3] **4** [7 7 8 6 5 9]

Et nyt pivot vælges i den større mængde

[1 2 3] **4** [7 7 [**8**] 6 5 9]

Den større delmængde sorteres

[1 2 3] **4** [7 7 6 5 < **8** < 9]



QuickSort

Et nyt pivot vælges i den mindre delmængde

1 2 3 4 [7 [7] 6 5] 8 [9]

Mængden sorteres

1 2 3 4 [7 5 6 <7<] 8 9

Et nyt pivot vælges i den mindre delmængde

1 2 3 4 [7 [5] 6] 7 8 9

Mængden sorteres

1 2 3 4 [<5<6 7] 7 8 9



QuickSort

- Algoritmen:
 - **Input:** En ikke tom mængde af tal, kaldet L f.eks. $\langle L_0, L_1, \dots, L_n \rangle$
 - **Output:** Sorteret mængde af tal $\langle L_0, L_1, \dots, L_n \rangle$ således $L_0 \leq L_1 \leq \dots \leq L_n$
- **Effektivitet**
 - **Best case:** $n \log n$ – Hvis listen allerede er delvist sorteret.
 - **Average case:** $n \log n$ – Typisk køretid ved tilfældige data.
 - **Worst case:** n^2 – Sker ved dårligt pivotvalg (f.eks. allerede sorteret liste)

Bemærk

- QuickSort er hurtigere end Bubble Sort og Insertion Sort for store datasæt.
- Det er dog **langsommere at verificere**, om en liste allerede er sorteret



QuickSort – Pseudo og opsummering

• Pseudokode for QuickSort

1. Vælg en pivot (typisk første, sidste eller midterste element).
2. Opdel listen i to dele:
3. Mindre end pivot
4. Større end pivot
5. Sorter begge dele rekursivt.
6. Saml delene til en fuldt sorteret liste.

• Opsummering af QuickSort

- Bruger **Divide and Conquer** – opdeler problemet i mindre dele og løser dem rekursivt.
- **Gennemsnitlig kompleksitet:** $n \log n$ – hurtigere end n^2 -sorteringer.
- **Worst case:** n^2 – sker hvis pivotvalg er dårligt (f.eks. allerede sorteret liste).
- **God til store datasæt** – hurtigere end Bubble Sort og Insertion Sort.



QuickSort – Pseudokode eksempel

- QuickSort kaldet
 - Simpel og ikke hukommelses optimeret version
 - Bruger flere lister / stakke

quicksort(A):

if (length of A ≤ 1)

return A

pivot:= A[0]

before:= New Stack of Data

after:= New Stack of Data

For i = 1 to Length of A

if A[i] < pivot then

before.push(A[i])

else

after.push(A[i])

End if

End For

tmpArray := New Array with Data from(Quicksort(before) + pivot + Quicksort(after))

return tmpArray

Hvis der kun er 0 eller 1 element er A allerede sorteret. Return derfor A

Opdeling
af array'et
i før og
efter pivot

Læg mærke til at her kalder algoritmen sig selv igen.
Dvs. rekursion



Søgning



Linear search

- En af de store fordele ved at have en sorteret liste, er at man hurtigere kan finde specifik data.
- Vi kender det fra oplag der er alfabetisk sorteret
 - Det er væsentlig hurtigere end at lede i tilfældigt arrangerede lister etc.
- Vi ser på et eksempel, og hvordan den virker
- For det meste er disse implementeret i forvejen, f.eks findes de i c#



Linear search

- Linear search er den mest simple



- Vi løber simpelt gennem hele listen indtil vores værdi er fundet
- $O(n)$, da vores element kan være sidst, eller slet ikke i listen
- Denne virker også på en usorteret liste



Linear search

- Linear search pseodokode:

```
Integer: LinearSearch(Data values[], Data target)
  For i = 0 To <length of values> - 1
    // See if this is the target.
    If (values[i] == target) Then Return i
    // See if we have passed where the target would be.
    If (values[i] > target) Then Return -1
  Next i
  // If we get here, the target is not in the array.
  Return -1
End LinearSearch
```

LinearSearch(L, target)

1. For hvert element i listen L:
 - a. Hvis elementet er lig med target:
 - Returnér elementets position
2. Hvis target ikke findes i listen:
 - Returnér "Ikke fundet"



Linear search

- Linear search i c#:

```
List<int> ints = new List<int> { 1, 2, 3 };  
int findList = ints.Find(x => x == 3);  
Console.WriteLine(findList.ToString());
```

- Returnere den første værdi der matcher prædikatet
 - (x=>x==3)
- Vil blot returnere 3, hvis værdien ikke findes giver den 0



Linear search

- List<T> har flere indbyggede der alle gør brug af linear search f.eks:
- .Exists – true/false hvis værdien findes
- .FindAll – Liste af alle elementer der matcher prædikat
- .FindLast - sidste element der matcher prædikat
- .FindIndex
 - tager op til 3 parameter start, Index, og count. Søger i gennem dette subset og returnere index på første fundne element
- .FindLastIndex
 - Ligesom findIndex, men finder blot det sidste der matcher



Linear search

- Alle disse liste funktioner har som sagt kompleksitet på $O(n)$
- Har dog den fordel at de fungerer på usorterede lister
- Hvis find operationer skal bruges flere gange, kan det give mening at først sortere listen og dernæst udføre operation
- Det er afhængig af use case hvad der giver mening.
 - Især størrelsen af listen spiller ind her!



Øvelse

- Lav en implementering af quicksort således at I kan sortere et array eller en liste
 - I kan finde mere info omkring QuickSort her: <http://en.wikipedia.org/wiki/Quicksort>
- Implementer 3 sortmetoder på din generiske liste:
 - QuickSort
 - BubbleSort
 - InsertionSort
- Hint til comparison af generiske typer på næste slide!
- Lav en fyldestgørende unit test af de 3 sort metoder
- Lav en in-place implementering af quicksort(VALGFRI)
 - Se på de næste valgfrie slides og
 - Lav en fyldestgørende test af inplace quicksort

øvelse 024



Hint comparison

- Ved at bruge den indbyggede comparer klasse

```
private IComparer<T> comparer;  
  
public GenericList()  
{  
    elements = new T[4];  
    comparer = Comparer<T>.Default;  
}
```

- Kan der sammenlignes med den standard comparer som returnere -1,0,1 !

`int IComparer<T>.Compare(Tx, Ty)`
Compares two objects and returns a value indicating whether one is less than, equal to, or greater than the other.



Valgfri slides

Optimeret Quicksort



QuickSort - Pseudo

- QuickSort kaldet
- Sorterer elementer fra "l" til "r"(inclusive) i et array A

```
quicksort(A, l, r):
```

```
    if l < r:
```

```
        p := partition(A, l, r)
```

```
        quicksort(A, l, p - 1)
```

```
        quicksort(A, p + 1, r)
```



QuickSort - Pseudo

- Partition kaldet

```
// left is the index of the leftmost element of the subarray
// right is the index of the rightmost element of the subarray (inclusive)
// number of elements in subarray = right-left+1
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

7 2 7 9 4 3 8 6 5 1



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

7 2 7 9 [4] 3 8 6 5 1



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

7 2 7 9 1 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 0

↪ 7 2 7 9 1 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

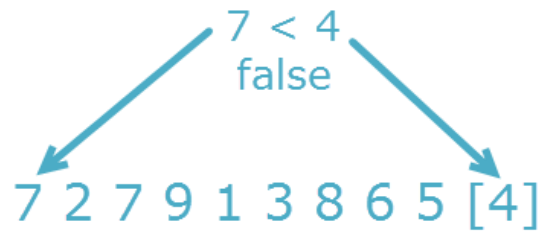
right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 0

i: 0



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

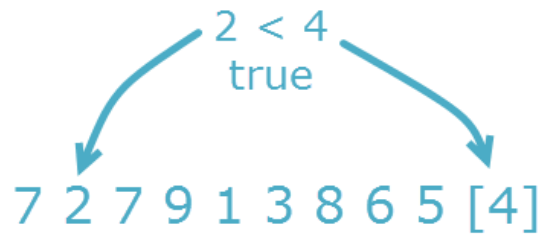
right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 0

i: 1



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 0

i: 1

2 7 7 9 1 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 1

i: 1 2 7 7 9 1 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

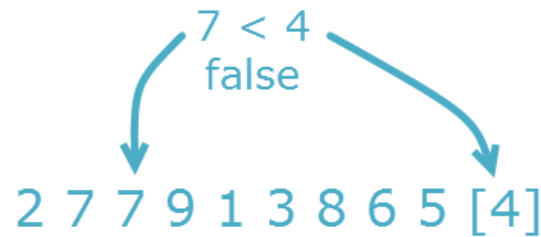
right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 1

i: 2



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

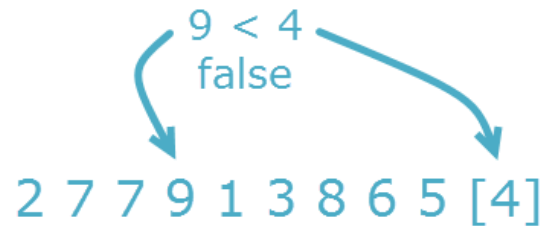
right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 1

i: 3



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

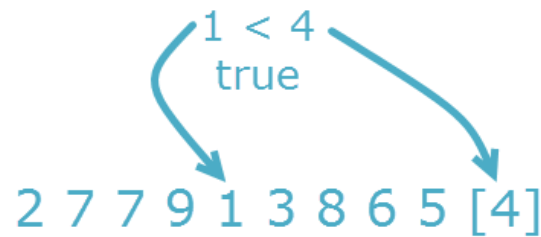
right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 1

i: 4



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 1

i: 4

2 1 7 9 7 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 2

i: 4

2 1 7 9 7 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

3 < 4

true

pivotValue: 4

storeIndex: 2

i: 5

2 1 7 9 7 3 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 2

i: 5

2 1 3 9 7 7 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 3

i: 5

2 1 3 9 7 7 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
  pivotIndex := choose-pivot(array, left, right)
  pivotValue := array[pivotIndex]
  swap array[pivotIndex] and array[right]
  storeIndex := left
  for i from left to right(inclusive)
    if array[i] < pivotValue
      swap array[i] and array[storeIndex]
      storeIndex := storeIndex + 1
  swap array[storeIndex] and array[right] // Move pivot to its final place
  return storeIndex
```

left: 0

right: 9

pivotIndex: 4 resten er false

pivotValue: 4

storeIndex: 3

i: 5

2 1 3 9 7 7 8 6 5 [4]



QuickSort - Partition

```
partition(array, left, right)
    pivotIndex := choose-pivot(array, left, right)
    pivotValue := array[pivotIndex]
    swap array[pivotIndex] and array[right]
    storeIndex := left
    for i from left to right(inclusive)
        if array[i] < pivotValue
            swap array[i] and array[storeIndex]
            storeIndex := storeIndex + 1
    swap array[storeIndex] and array[right] // Move pivot to its final place
    return storeIndex
```

left: 0

right: 9

pivotIndex: 4

pivotValue: 4

storeIndex: 3

i: 5

